POLYMORPHISM

→It is taken from the greek words

→It means that the same function name can be used for different types.

Example-1:

```
#polymorphism with functions and objects
class Tomato:
   def type(self):
      print("vegetable")
   def color(self):
      print("red")
class Apple:
   def type(self):
      print("fruit")
   def color(self):
      print("green")
def func(obj):
   obj.type()
   obj.color()

t=Tomato()
a=Apple()
func(t)
func(a)
```

Example-2:

```
#polymorphism with class method
class india():
   def capital(self):
      print("new delhi")
   def language(self):
      print("eng and hindi")
class USA():
   def capital(self):
      print("washing ton")
   def language(self):
      print("english")
i=india()
u=USA()
for country in(i,u):
   country.capital()
   country.language()
```

method overloading:

writing one or more methods with same name but difference in the arguments is called “method overloading”.

arguments different will be based on a no.of arguments and type of arguments.

It is used in a single class.

It is used to write code clarity as well as reduce complexity.

Example:

```
class Area():
   def find_area(self,a=None,b=None):
      if a!=None and b!=None:
         print("area of rectangle:",(a*b))
      elif a!=None:
         print("Area of square:",(a*a))
      else:
         print("nothing is find")
obj1=Area()
obj1.find_area()#nothing is find
obj1.find_area(10)#100
obj1.find_area(23,34)#782
```

method overriding:

writing one or more methods with same name and same parameters is called method overriding.

it is implemented with inheritance.

it is mostly used for memory reducing process.

Example:

```
class Rectangle():
   def __init__(self,length,breath):
      self.length=length
      self.breath=breath
   def getArea(self):
      print(self.length*self.breath,"is area of rectangle")
class Square(Rectangle):
   def __init__(self,side):
      self.side=side
      Rectangle.__init__(self,side,side)
   def getArea(self):
      print(self.side*self.side,"is area of square")
s=Square(4)
r=Rectangle(23,34)
s.getArea()
r.getArea()
```

method overloading with multipledisptch module

method overloading is achievable in single class not in inherited classes.

For achieving this we need to import dispatch from multipledispatch module.

i.e., from multipledispatch import dispatch

This multiple dispatch is done for both functions & methods . for controlling their calling based on their arguments or parameters.

E.g.,:- (on static methods)

```
 from multipledispatch import dispatch

class cl:

    @staticmethod

    @dispatch(str)

    def m_1(a):

        return a

    @staticmethod

    @dispatch( str, str)

    def m_1(a,b):

        return a+b

c=cl()

print(c.m_1("hello","world")) # prints 1

print(c.m_1("have a nice day")) # prints 3
```

E.g.,:-(on methods)

```
from multipledispatch import dispatch

class example:
```

```
  @dispatch(int, int)
  def add(self, a, b):
    x = a+b
    return x
  @dispatch(int, int, int)
  def add(self, a, b, c):
    x = a+b+c
    return x
obj = example()
print (obj.add(10,20,30))
print (obj.add(10,20))
```